



COM410 Programming in Practice

T7. Recursion



Recursion

What is recursion?

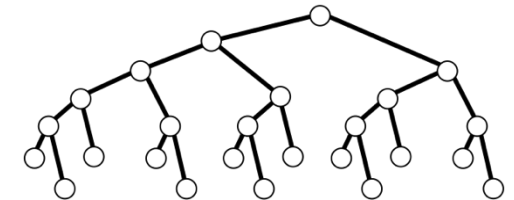
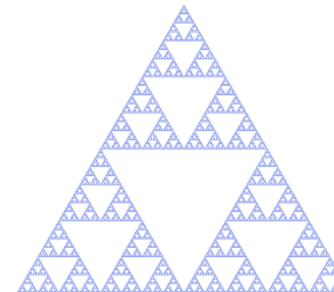
- When something is specified in terms of itself

Why learn recursion?

- It represents a new mode of thinking
- Provides a powerful programming paradigm
- Can give insight into nature of computation

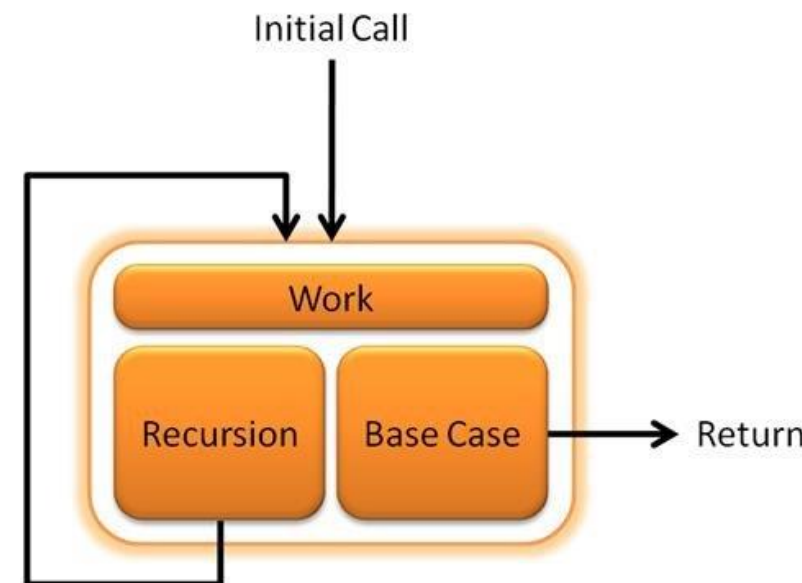
Naturally self-referential computational artifacts

- File systems with folders containing folders
- Fractal graphics patterns
- Divide and conquer algorithms



What is Recursion (in programming)?

- Recursion happens when a method calls itself *within its own definition*
- A method that calls itself is a **recursive method**
- Two main parts of a recursive method:
 1. **Base Case**: important as without it the method would never stop (the smallest identical problem that has a known solution)
 2. **Recursive Call**: the recursive invocation of a recursive method (solving the smaller identical problem)



Example: The Countdown

- Recursive method to print out all integers from a given starting value down to one

```
public static void countDown(int start)
{
    System.out.println(start);
    if (start > 1) {
        countDown(start - 1);
    }
}
```

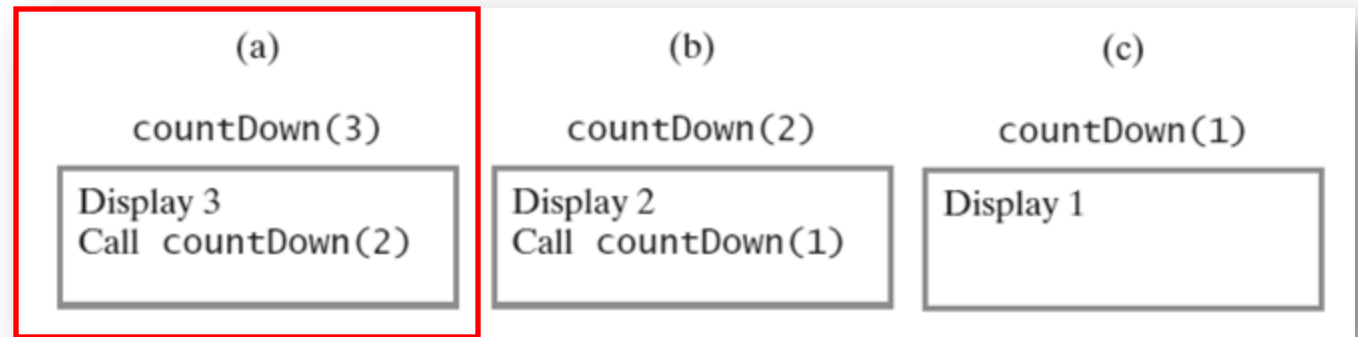
Base Case

Recursive Call

Tracing a Recursive Method

- Trace a call to `countDown(3)` ;
- The argument 3 is copied into the parameter `start` and the following statements of the method are executed:

```
System.out.println(3) ;  
if (3 > 1)  
    countDown(3 - 1) ;
```

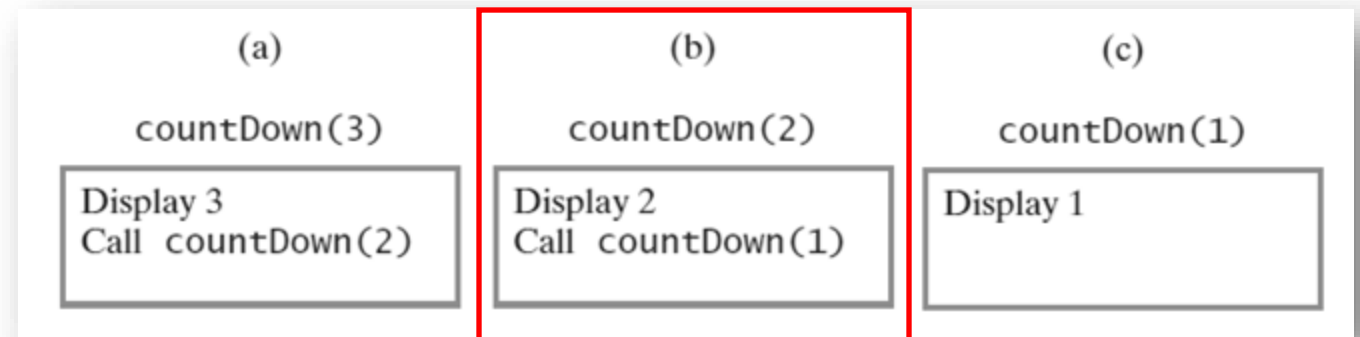


a) A line containing “3” is output and the recursive call to `countDown(2)` occurs

Tracing a Recursive Method

- Execution of the original call to `countDown(3)` is **suspended** until the result of `countDown(2)` is known (even though nothing happens after the recursive call)
- Continuing the trace, `countDown(2)` executes the following statements:

```
System.out.println(2);  
if (2 > 1)  
    countDown(2 - 1);
```

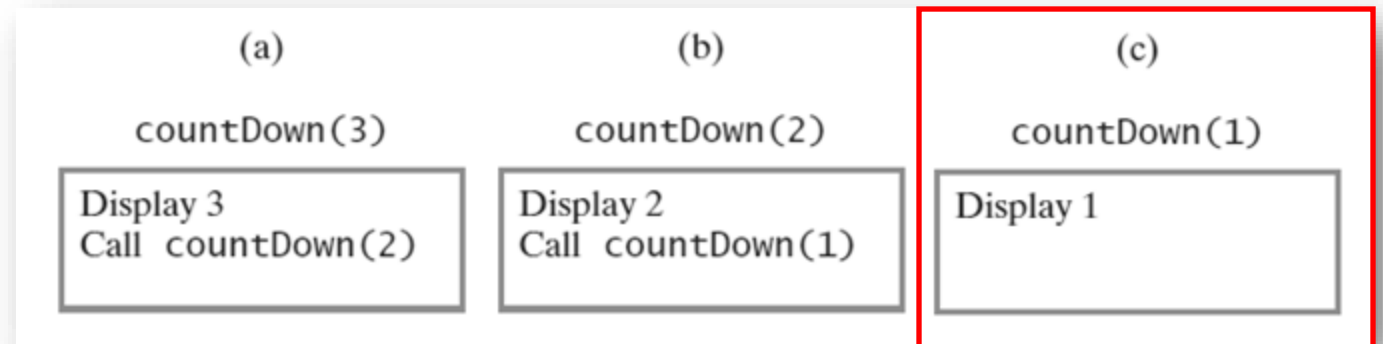


b) A line containing “2” is output and the recursive call to `countDown(1)` occurs

Tracing a Recursive Method

- Continuing the trace, `countDown(1)` executes the following statement:

```
System.out.println(1) ;  
if (1 > 1)
```



- c) A line containing “1” is output and no other recursive calls occur

Using Recursion

- Compute the sum $1 + 2 + \dots + n$ for any integer $n > 0$
- The given input for the problem is integer n
- What small value of n results in a sum that you know immediately? (i.e. what is the base case? How do we know when we are finished?) One possible answer is 1 (if $n = 1$, sum = 1)
- What should the smaller version of the task be? Compute the sum $1 + \dots + (n - 1)$ (adding n to this computation will solve the original problem)
- Leads to the following definition of `sumOf ()`

$$\sum_{i=1}^n i$$

```
public static int sumOf(int n) {  
    if (n == 1) return 1;  
    else return n + sumOf(n - 1);  
}
```


Tracing `sumOf()`

- Suppose we invoke the method with the statement: `sumOf(3);`
- The computation occurs as follows:
 - a) `sumOf(3)` is $3 + \text{sumOf}(2)$ (`sumOf(3)` suspends execution and `sumOf(2)` begins)
 - b) `sumOf(2)` is $2 + \text{sumOf}(1)$ (`sumOf(2)` suspends execution and `sumOf(1)` begins)
 - c) `sumOf(1)` returns 1 (base case is reached so suspended execution resumes with most recent activation)
 - d) `sumOf(2)` returns $2 + 1 = 3$
 - e) `sumOf(3)` returns $3 + 3 = 6$

```
public static int sumOf(int n) {  
    if (n == 1) return 1;  
    else return n + sumOf(n - 1);  
}
```

Scenario

- Demonstration of recursive programming
 - Create a new Java project called **Recursion**. In the project, add a new class **SumOf** in a file called **sumOf.java** containing a method **sumOf ()** that takes a single integer parameter and returns the sum of all positive integers up to and including that value.
 - Verify the **sumOf ()** method by providing code to a **main ()** method in the **SumOf** class that prints the result of calls to **sumOf ()** for values from 1 to 10.

Unwise Recursion

- **Fibonacci Numbers** (Fibonacci Sequence)
- Created in 13th Century by Leonardo Fibonacci who proposed a sequence of integers to model the number of descendants of a pair of rabbits
- Models many natural phenomena (found a lot in art and architecture)



Let $F_n = F_{n-1} + F_{n-2}$ for $n > 1$ with $F_0 = 0$ and $F_1 = 1$.

n	0	1	2	3	4	5	6	7	8	9	10	11	12	13	...
F_n	0	1	1	2	3	5	8	13	21	34	55	89	144	233	...

- First two terms in the sequence are 0 and 1
- Each subsequent term in the sequence is the sum of the preceding two terms

Unwise Recursion

- A recursive algorithm is a *tempting* way to generate the sequence:

```
public long fib(int n) {  
    if (n == 0) return 0;  
    if (n == 1) return 1;  
    return fib(n - 1) + fib(n - 2);  
}
```

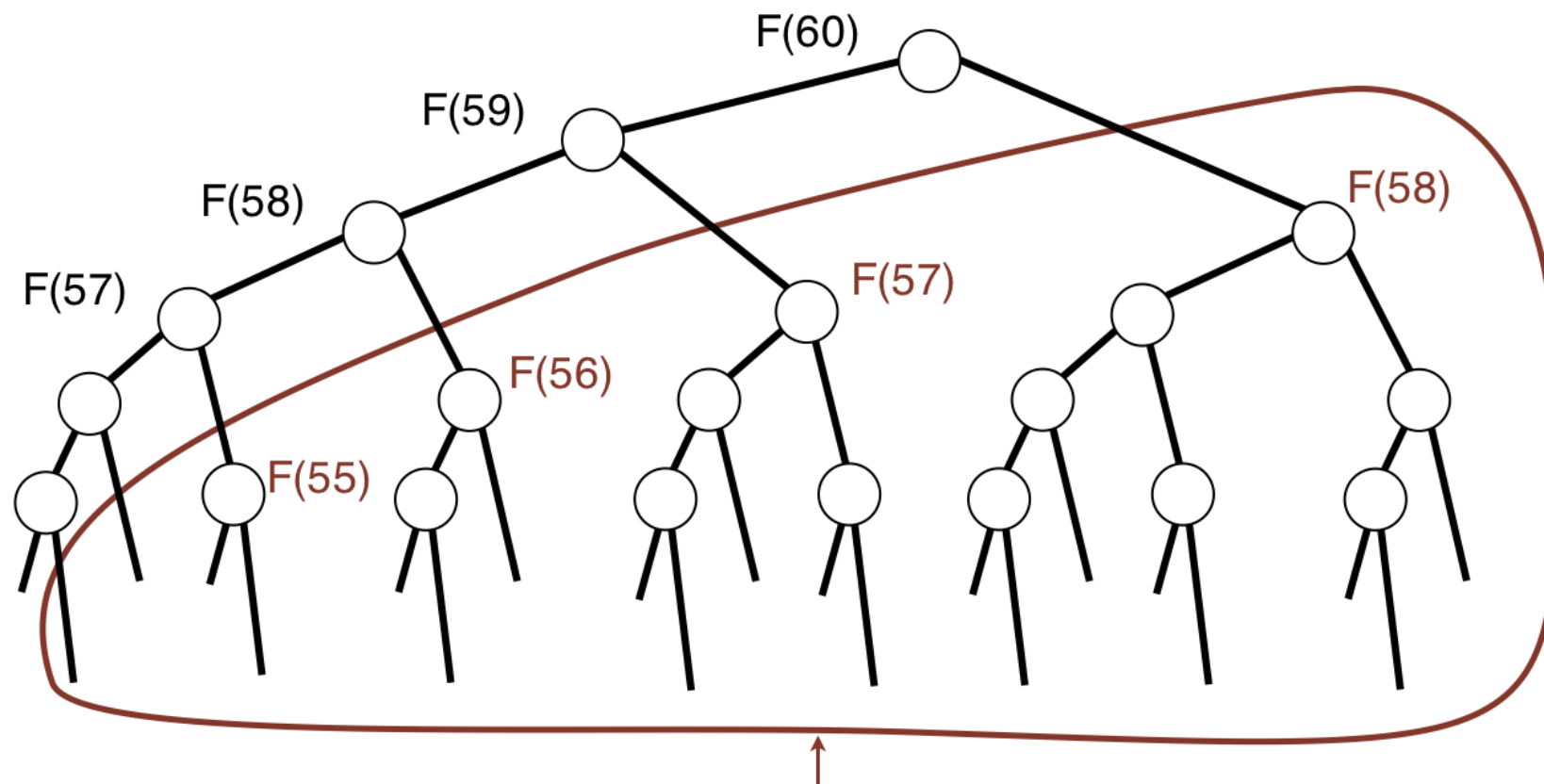
- Trouble is same recursive calls made repeatedly!
 - A call to `fib(n)` invokes `fib(n-1)` then `fib(n-2)`
 - Call to `fib(n-1)` has to compute `fib(n-2)` so same Fibonacci number is computed twice
 - Call to `fib(n-1)` invokes `fib(n-3)` as well but two previous calls to `fib(n-2)` each invoke `fib(n-3)` so `fib(n-3)` is computed 3 times!

-

Unwise Recursion

- Let C_n be the number of times $F(n)$ is called when calculating $F(60)$

n	C_n
60	1
59	1
58	2
57	3
56	5
55	8
	...
0	$>2.5 \times 10^{12}$



Exponentially wasteful to recompute all these values.
(trillions of calls on $F(0)$, not to mention calls on $F(1)$, $F(2)$,...)

Scenario

- **Demonstration of unwise recursion**
- In the **Recursion** project, add a new class **Fibonacci** in a file **Fibonacci.java**. Implement the method **fibonacci(n)** that takes a single integer parameter and returns the corresponding Fibonacci term, calculated using recursion.
- In **main()**, call the method with **fibonacci(20)**. Now, prove that the number of calls to **fibonacci(n)** for descending values of **n** to 1 themselves form a Fibonacci sequence,

such that...

- fibonacci(20)** is called **1** time
- fibonacci(19)** is called **1** time
- fibonacci(18)** is called **2** times
- fibonacci(17)** is called **3** times
- fibonacci(16)** is called **5** times
- fibonacci(15)** is called **8** times, etc...